LESSON 09

加起来 += 数一数

累加和计数

学完以后，一串数读完，程序能告诉你总共是多少、一共有几个。

一串成绩，总分多少？几个及格？

输入 4 个成绩

80 59 60 30

上节课的程序会一个一个输出：80，再 60。
可是这次我们只要**一个答案**：总分 229，及格 2 人。

每一轮只看得到手里这一个数，前面的数已经过去了。
所以要准备一个盒子，把前面的结果一路记下来，
循环走完，答案就在盒子里。

total 是一个存钱罐

sum.cpp

```
 7      int total = 0;
 8      for (int i = 1; i <= n; i++) {
 9          total += i;
10      }
11      cout << total << endl;
```

输入 3，屏幕上

6

存钱罐先是空的。每一轮把 i 放进去，罐子里的钱越来越多。

total += i 就是 total = total + i，第 6 课学过。

输入 3：罐子里的钱怎么变

i 是 1	0 + 1	total 变成 1
i 是 2	1 + 2	total 变成 3
i 是 3	3 + 3	total 变成 6
i 是 4	4 <= 3 不成立	循环结束，输出 6

total 里放的，一直是“到目前为止的和”。

每一轮的 total 都接着上一轮，所以它要活得比循环长。

三句话，三个位置

循环前面

`int total = 0;`

准备空罐子，只做一次。

循环里面

`total += i;`

每轮放进一个数。

循环后面

`cout << total;`

全部放完才打开看。

累加的错误，大多是这三句放错了位置。

写完先看一眼：这句该运行一次，还是每轮都运行？

猜一猜：清零写进了循环

sum.cpp

```
 7      int total = 0;
 8      for (int i = 1; i <= n; i++) {
 9          total = 0;
10          total += i;
11      }
12      cout << total << endl;
```

屏幕上

3

前面放的都被倒掉了，最后只剩 `i` 是 `3` 的那一次。

准备罐子只做一次，要写在循环前面。

小测：cout 搬进循环，输入 3 输出什么？

```
total += i;  cout << total << endl;
```

A. `1 6`

B. `2 1 3 6`

C. `3 1 2 3`

D. `4 0 1 3`

答对了！每轮加完就输出一次，屏幕上是到目前为止的和。

动手改一改

从 sum 开始。不是每个数都放进罐子，加一个筛子。

偶数的和

读入 n，是 0 到 1000 的整数。输出 1 到 n 里所有偶数的和。

输入 6（2 + 4 + 6）

```
12
```

输入 1（没有偶数）

```
0
```

动手改一改

也从 sum 开始。这回放进罐子的，是读进来的数。

全班总分

先读入 n（0 到 100），再读入 n 个成绩（0 到 100），都是整数。输出总分。

输入 3，再输入 80 85 90

```
255
```

输入 0

```
0
```

count 是一个计数器

count.cpp

```
7       int count = 0;
8       for (int i = 1; i <= n; i++) {
9           if (i % 3 == 0) {
10              count++;
11          }
12      }
13      cout << count << endl;
```

输入 10，屏幕上

3

像门口数人的计数器：来一个，按一下，只加 1。

位置和累加一样：循环前准备，循环里加，循环后输出。

输入 10：3 的倍数，一边加一边数

碰到 3	**total 加 3，count 加 1**	total 3，count 1
碰到 6	**total 加 6，count 加 1**	total 9，count 2
碰到 9	**total 加 9，count 加 1**	total 18，count 3

累加放进去的是**这个数本身**，计数每次**只加 1**。

同一个 if 里，两件事可以一起做，最后输出 18 和 3。

小测：count 写成这样，输入 10 输出几？

```
if (i % 3 == 0) { count += i; }
```

1 3

2 9

3 18

4 10

答对了！3 + 6 + 9 是 18。这样算的是总和，不是个数。

一个也没有，才说 No

none.cpp

```
10                 cout << i << endl;
11                 count++;
12             }
13         }
14         if (count == 0) {
15             cout << "No" << endl;
16         }
```

输入 4

```
2
4
```

输入 1

```
No
```

上节课的 else 只知道这一轮。count 记得全部：
循环走完它还是 0，说明一个也没碰到，这时候说一次 No。

动手改一改

从 none 开始，改筛子，改最后说的话。

7 的倍数在哪？

读入 n，是 0 到 100 的整数。输出 1 到 n 里 7 的倍数，每个数占一行。一个也没有，就输出一次 None。

输入 15

```
7
14
```

输入 6

```
None
```

动手改一改

从 count 开始。这回要数的，是读进来的数。

几个人及格?

先读入 n（0 到 100），再读入 n 个成绩（0 到 100），都是整数。60 分及以上算及格，输出及格人数。

输入 4，再输入 80 59 60 30

```
2
```

输入 2，再输入 0 59

```
0
```

猜一猜：平均分是多少？

average.cpp

```
13      cout << total / n << endl;
```

屏幕上

83

这是第 2 课的整数除法：两个 int 相除，小数被去掉了。
总分是整数没问题，问题出在除的时候。

先乘 1.0，再除

average.cpp

```
13      cout << total * 1.0 / n << endl;
```

屏幕上

83.6667

total * 1.0 得到一个带小数的数，再除就不会丢掉小数。
第 2 课的 7.0 / 2 也是这个道理。

n 要从 1 开始。总共 0 个人时不能算平均分：
整数除以 0 没有正确答案，程序可能直接退出，也可能给出错的数。

n 是 1 到 100，成绩是 0 到 100，都是整数。

读报错：同一个盒子准备了两次

sum.cpp（只阅读）

```
 7      int total = 0;
 8      int total = 0;
```

redefinition 的意思是：这个盒子已经准备过了，又准备了一次。下面的 note 指着第 7 行，第一次准备在那里。

```
main.cpp:8:9: error: redefinition of 'total'
main.cpp:7:9: note: previous definition is here
```

同一层大括号里，一个名字只能准备一个盒子。

想再用它，直接写 total，前面不用再写 int。删掉第 8 行就好。

忘了放 0：没有 error，答案却怪

sum.cpp（只阅读）

```
7       int total;
8       for (int i = 1; i <= n; i++) {
9           total += i;
10      }
```

盒子准备好了，可是没放东西。里面是什么，没人知道。

输入 3，有时碰巧是 6，有时是 -148750242 这样的怪数，换一台电脑又不一样。

所以程序碰巧答对，不代表它是对的。

盒子准备好就放一个数，这样的错误就不会有。

找错：哪一行有问题？

sum.cpp（只阅读）

```
1       int total = 0;
2       for (int i = 1; i <= n; i++) {
3           total =+ i;
4       }
5       cout << total << endl;
```

这段程序想算 1 到 n 的和。
它没有报 error，可是输入 3，
屏幕上是 3，不是 6。
点你认为有问题的那一行。

点代码里的一行

罐子太小：int 装不下

big.cpp

```
7       long long total = 0;
8       for (int i = 1; i <= n; i++) {
9           total += i;
10      }
11      cout << total << endl;
```

输入 100000，屏幕上

5000050000

常见的 int 最大约 21 亿，1 到 100000 的和超过了它。

用 int 时结果不可靠：我们试的电脑上是 705082704，换一台可能不同，也没有 error。

long long 是更大的整数盒子，和有可能很大时用它。

自己挑战

拆开一个数

读入一个整数 n，是 1 到 1000000000。第一行输出 n 有几位，第二行输出各位数字的和。

输入 1234，输出

```
4
10
```

输入 7，输出

```
1
7
```

先自己写，卡住了再翻下一页。

卡住了？一次看一条提示

提示 1　事先不知道有几位，想想第 7 课“倒着念数字”是怎么一位一位拿的。

提示 2　要记住两件事，所以要准备两个盒子。它们一开始都放几？

提示 3　每拿到一位，一个盒子加这一位，另一个盒子加 1。

造一个小账本

一直读数字，读到 0 才停，最后告诉我一个结果。主题你自己定。

先做到

用 while 一直读，把读到的数加起来，读到 0 输出总数。

选你喜欢的

零花钱账本、走路步数、吃了几颗糖、游戏得分，都可以。

还想更厉害

同时数一数记了几笔，再算出平均每笔多少。

没有标准答案。写之前说清想要的结果，写完试试最小和最大的输入。

和同桌换着玩

1

换着运行

运行同桌的账本，先记几笔，再试试一上来就输入 `0`

2

说一个喜欢的

告诉同桌，你最喜欢这个程序哪里

3

提一个想法

问问同桌：还可以多告诉我一个什么结果？

听完再改一版。想一想、做出来、试一试、分享、再改，程序就是这样越做越好的。

说得清吗?

Q1 **准备、更新、输出，分别写在循环的什么位置?**

准备在循环前，更新在循环里，输出在循环后。

Q2 **`total += i` 和 `count++`，每轮加的有什么不同?**

`total` 加的是这个数本身，`count` 每次只加 `1`。

Q3 **`251 / 3` 为什么是 `83`? 怎样得到小数?**

两个 `int` 相除会去掉小数，先乘 `1.0` 再除就能留下。

速查卡

```
int total = 0;  int count = 0;
```

`total += i;`	累加：放进这个数本身
`count++;`	计数：只加 1
`count == 0`	循环后判断：一个也没有
`total * 1.0 / n`	留下小数，n 不能是 0
`long long`	和可能超过约 21 亿时用它

这些后面再学

最大值

一串数里，记住最大的那个

循环里放循环

一行重复，一整行也重复

数组

把一串数都存下来，以后还能用

今天学会累加和计数，一个盒子记下全过程，循环走完答案就有了。

谢谢！

喜欢这节课？欢迎来 GitHub 和我交流。

 cpp-for-youth

下一课：最大值与最小值